

阿里云前端技术专家-AI领域

一面（技术基础） 1-6

1. 请简述 SSE (Server-Sent Events) 与 WebSocket 的区别，以及在AI对话场景下为什么通常选择 SSE?
2. 在处理 LLM 返回的 Markdown 流时，如何解决代码块或公式被截断导致的渲染闪烁问题?
3. 请手写一个简单的流式数据解析器，模拟处理 Fetch API 返回的 ReadableStream。
4. 谈谈 Promise.all、Promise.allSettled 和 Promise.race 的区别，并举例 AI 场景下的应用。
5. 如何实现一个对话界面的“自动滚动到底部”功能，并兼顾用户手动向上滚动查看历史记录的体验?
6. 介绍一下 TypeScript 中的高级类型（如 Record, Partial, Pick），并说明在定义AI接口响应时的作用。

二面（深入原理与项目） · 1-8

1. 在构建 AI Chat 应用时，长列表的性能优化有哪些核心方案?
2. 请详细说明 RAG（检索增强生成）前端链路中，如何处理文档上传、切片与向量化的交互流程?
3. 谈谈你对 Prompt Engineering的理解，前端如何通过工程化手段提升 Prompt 的质量和安全性?
4. 如何设计一个通用的 AI 组件库 (AI UI Kit)? 需要考虑哪些特殊组件?
5. 在AI 应用中，如何优化首屏加载速度，特别是涉及大型 WebAssembly 模块（如本地向量化模型）时?
6. 解释一下 Function Calling（函数调用）的原理，前端如何配合模型完成一次完整的工具调用?
7. 你们项目中是如何处理 AI生成内容的持久化和多端同步的?
8. 谈谈 Web Worker 在 AI前端应用中的实际应用场景。

三面（架构与综合能力） 1-6

- 1.如果让你从零设计一个企业级 AI Copilot 的前端架构，你会如何考虑技术选型和模块划分?
- 2.AI领域技术迭代极快，你如何保证前端团队在快速跟进新技术的同时，维持系统的稳定性?
 1. 在 AI 落地业务的过程中，你遇到过哪些关于“用户体验”的挑战？你是如何解决的？
 2. 谈谈 Web AI 的未来趋势，例如 WebGPU 对前端AI应用会带来什么改变？
 3. 如何评估一个 AI前端项目的成功？你会关注哪些核心指标？
 4. 聊聊你在过去一年中遇到的最难的一个 AI相关技术问题，你是如何排查并解决的？

一面（技术基础）

1. 请简述 SSE（Server-Sent Events）与 WebSocket 的区别，以及在 AI 对话场景下为什么通常选择 SSE？

难度：★★

考点：网络协议、流式传输

答案要点：

SSE 是一种基于 HTTP 的单向推送技术，而 WebSocket 是全双工的二进制协议。在 AI 场景下，SSE 因其轻量级和原生支持流式解析而成为首选。

- 协议层：SSE 基于标准 HTTP 协议，支持自动重连，WebSocket 则是独立的 TCP 协议。
- 数据格式：SSE 传输的是文本流（UTF-8），WebSocket 支持二进制和文本。
- 场景匹配：LLM 输出是单向流式的，SSE 建立连接简单，且能更好地穿透防火墙和代理。
- 资源消耗：SSE 在服务端压力相对较小，且支持 HTTP/2 多路复用。

常见坑：

- 忽略了 SSE 在 HTTP/1.1 下有 6 个连接限制的问题。
- 混淆了长轮询（Long Polling）与 SSE 的区别。

追问：

- 如果需要实现用户在生成过程中中断对话，SSE 应该如何处理？
- 在 HTTP/2 环境下，SSE 的连接限制还存在吗？

2. 在处理 LLM 返回的 Markdown 流时，如何解决代码块或公式被截断导致的渲染闪烁问题？

难度：★★

考点：流式渲染、DOM 性能

答案要点：

流式渲染的核心在于增量更新与状态保持，通过缓冲区机制或解析器的状态机来处理未闭合的标签。

- 增量解析：使用支持流式的 Markdown 解析器（如 markdown-it 的插件或自定义逻辑），记录当前解析状态。
- 缓冲区处理：对代码块（`````）或公式（ $...$ ）进行正则匹配，若未发现闭合标签，则暂存或进行特殊占位处理。
- 视觉平滑：利用 CSS 动画或打字机效果掩盖渲染过程中的布局跳动。
- 虚拟 DOM 优化：在 React/Vue 中，确保 key 值的稳定性，避免整个列表重新渲染。

常见坑：

- 每次收到 chunk 都全量重新解析整个字符串，导致性能崩溃。

- 忽略了 HTML 标签被从中间截断导致的非法 DOM 结构。

追问：

- 如果返回的内容包含 LaTeX 公式，你会选择哪个库来渲染？

3. 请手写一个简单的流式数据解析器，模拟处理 Fetch API 返回的 ReadableStream。

难度：★★

考点：JavaScript 异步迭代、Stream API

答案要点：

利用 `response.body.getReader()` 循环读取数据块，并通过 `TextDecoder` 将 `Uint8Array` 转换为字符串。

- 获取 Reader：通过 `getReader()` 获取流的读取器。
- 循环读取：使用 `while(true)` 循环配合 `await reader.read()`。
- 解码处理：使用 `TextDecoder` 处理字节流，注意处理跨 chunk 的字符截断。
- 异常处理：try-catch 捕获网络中断，并确保调用 `reader.releaseLock()`。

常见坑：

- 忘记处理 `done` 状态导致死循环。
- 直接 `toString()` 导致中文字符在 chunk 边缘被截断产生乱码。

代码示例：

代码块

```
1  async function fetchStream(url) {
2    const response = await fetch(url);
3    const reader = response.body.getReader();
4    const decoder = new TextDecoder();
5    while (true) {
6      const { done, value } = await reader.read();
7      if (done) break;
8      const chunk = decoder.decode(value, { stream: true });
9      console.log('Received chunk:', chunk);
10   }
11 }
```

4. 谈谈 Promise.all、Promise.allSettled 和 Promise.race 的区别，并举例 AI 场景下的应用。

难度：★

考点：异步编程

答案要点：

这三个方法用于处理多个并发异步任务，区别在于对“成功”和“失败”的判定逻辑不同。

- `Promise.all`: 全部成功才返回，一个失败即整体失败。适用于 AI 批量生成摘要。
- `Promise.allSettled`: 无论成功失败，返回所有任务的结果。适用于并行调用多个模型并对比结果。
- `Promise.race`: 返回最快的结果。适用于多模型竞速，取响应最快的那个。
- 错误处理: `allSettled` 不需要 `catch` 每一个子项，而 `all` 必须处理。

常见坑:

- 在 `Promise.all` 中忘记处理 `catch`，导致一个请求失败影响全部。

追问:

- 如果要实现一个带并发限制（如最多同时请求 3 个模型）的函数，你会怎么写？

5. 如何实现一个对话界面的“自动滚动到底部”功能，并兼顾用户手动向上滚动查看历史记录的经验？

难度: ★★

考点: DOM 操作、用户体验

答案要点:

通过监听滚动位置判断用户意图，只有当用户处于底部附近时才触发自动滚动。

- 阈值判断: 计算 `scrollHeight - scrollTop - clientHeight`，若小于某个像素值（如 50px），判定为在底部。
- 滚动锁定: 用户手动向上滚动超过阈值时，禁用自动滚动。
- 滚动平滑: 使用 `scrollTo({ behavior: 'smooth' })` 或 `scrollIntoView`。
- 性能优化: 对 `scroll` 事件进行节流处理。

常见坑:

- 强制自动滚动导致用户无法查看历史消息，产生“抢夺滚动条”的糟糕体验。
- 图片加载导致高度变化，未重新计算滚动位置。

追问:

- 如果对话内容非常多（上万条），如何优化长列表的渲染性能？

6. 介绍一下 TypeScript 中的高级类型（如 `Record`, `Partial`, `Pick`），并说明在定义 AI 接口响应时的作用。

难度: ★

考点: TypeScript 基础

答案要点:

高级类型能提升代码的健壮性和复用性，特别是在处理模型返回的复杂且不确定的 JSON 数据时。

- `Record`: 定义键值对映射，如 `Record<string, ModelConfig>`。

- Partial：将属性变为可选，适用于模型增量更新配置。
- Pick/Omit：从复杂对象中提取或剔除属性，用于精简前端展示模型。
- 联合类型与类型守卫：处理模型返回的不同 message 类型（text/image/tool_call）。

常见坑：

- 过度使用 `any` 导致类型系统失效。

追问：

- 如何根据模型返回的 `type` 字段自动推断出 `content` 的具体类型？

二面（深入原理与项目）

1. 在构建 AI Chat 应用时，长列表的性能优化有哪些核心方案？

难度：★★★

考点：虚拟列表、渲染性能

答案要点：

针对 AI 场景下内容动态增长、高度不固定的特点，需要结合虚拟滚动与动态高度计算。

- 虚拟列表（Virtual List）：只渲染可视区域内的 DOM，通过 `react-window` 或自定义实现。
- 动态高度缓存：对已渲染的消息进行高度测量并缓存，避免滚动时抖动。
- 渲染分帧：利用 `requestAnimationFrame` 或 `MessageChannel` 将大量 DOM 插入操作分解。
- 减少重绘：使用 `contain: strict` 或 `will-change` 优化 CSS 渲染性能。

常见坑：

- 虚拟列表在快速滚动时出现白屏。
- 消息中包含图片或 Markdown 导致高度在渲染后发生变化，未及时更新缓存。

追问：

- 如果消息中包含复杂的代码高亮和数学公式，如何进一步优化渲染耗时？

2. 请详细说明 RAG（检索增强生成）前端链路中，如何处理文档上传、切片与向量化的交互流程？

难度：★★★

考点：AI 工程化、RAG 流程

答案要点：

前端在 RAG 中主要负责文件预处理、进度反馈以及检索结果的展示。

- 文件预处理：在前端进行格式校验、大小限制，甚至利用 Web Worker 进行初步的文本提取。
- 切片策略反馈：提供可视化界面让用户配置 chunk size 和 overlap，并实时预览切片效果。

- 异步状态管理：由于向量化耗时较长，需通过 WebSocket 或长轮询实时展示索引进度。
- 引用溯源：在 AI 回复中展示引用片段，点击可跳转至原文档的具体位置（高亮显示）。

常见坑：

- 大文件上传未做分片，导致网络超时。
- 忽略了 PDF 等复杂格式在前端解析时的乱码问题。

追问：

- 如何在前端实现 PDF 内部特定文本的精准定位和高亮？

3. 谈谈你对 Prompt Engineering 的理解，前端如何通过工程化手段提升 Prompt 的质量和安全性？

难度：★★

考点：Prompt 工程、安全性

答案要点：

Prompt 工程是通过结构化输入引导模型输出预期结果的技术，前端可以通过模板化和预处理来增强。

- 模板化管理：将 System Prompt 与 User Input 分离，前端负责组装变量，避免用户直接接触核心指令。
- 结构化输出：通过 Prompt 强制模型返回 JSON，前端利用 JSON Schema 进行校验。
- 敏感词过滤：在发送前进行前端脱敏或合规性检查。
- Few-shot 注入：根据场景动态从本地缓存或向量库中检索示例，拼接到 Prompt 中。

常见坑：

- 将敏感逻辑写在前端 Prompt 中，容易被用户通过控制台截获。
- 忽略了 Token 长度限制，未在前端做截断处理。

追问：

- 什么是 Prompt Injection（提示词注入）？前端能做哪些防御？

4. 如何设计一个通用的 AI 组件库（AI UI Kit）？需要考虑哪些特殊组件？

难度：★★★

考点：组件化设计、抽象能力

答案要点：

AI 组件库不同于传统 UI 库，它更侧重于内容呈现的灵活性和交互的异步性。

- 核心组件：ChatContainer（容器）、MessageItem（消息条目）、InputArea（支持多模态输入）、ThoughtChain（思维链展示）。
- 渲染插件化：支持 Markdown、LaTeX、Mermaid 图表、代码沙箱等插件动态加载。
- 交互抽象：封装流式输出逻辑、停止生成、重新生成、反馈（点赞/踩）等通用行为。

- 主题与适配：支持暗黑模式，适配移动端窄屏对话体验。

常见坑：

- 组件耦合了具体的 API 调用逻辑，导致难以复用。
- 忽略了无障碍访问（A11y），盲人阅读器难以处理流式更新的内容。

追问：

- 如果来实现一个类似 Artifacts 的侧边栏预览功能，组件架构该如何设计？

5. 在 AI 应用中，如何优化首屏加载速度，特别是涉及大型 WebAssembly 模块（如本地向量化模型）时？

难度：★★★

考点：性能优化、WASM

答案要点：

针对大型资源，应采用分级加载、缓存策略及多线程技术。

- 资源预加载：利用 `<link rel="preload">` 或 Service Worker 预缓存 WASM 文件。
- 实例化优化：使用 `WebAssembly.instantiateStreaming` 提升编译速度。
- 离线存储：将模型文件存储在 IndexedDB 中，避免重复下载。
- 任务卸载：将 WASM 运行在 Web Worker 中，避免阻塞主线程 UI 响应。

常见坑：

- WASM 文件过大导致首屏 FCP 时间过长。
- 忽略了不同浏览器对 WASM 内存限制的差异。

追问：

- 如果用户网络极差，如何降级处理本地模型功能？

6. 解释一下 Function Calling（函数调用）的原理，前端如何配合模型完成一次完整的工具调用？

难度：★★★

考点：Agent、Function Calling

答案要点：

Function Calling 是模型决定调用外部工具的过程，前端负责执行具体的本地逻辑并反馈结果。

- 定义 Schema：前端定义工具函数的入参规范（JSON Schema）并传给模型。
- 状态识别：解析模型返回的 `finish_reason: 'tool_calls'`，提取函数名和参数。
- 本地执行：前端根据函数名执行对应的 JS 逻辑（如查询天气、搜索文档）。
- 结果回填：将执行结果再次发送给模型，由模型总结并输出最终回答。

常见坑：

- 模型生成的参数格式不符合预期，前端未做校验导致报错。

- 循环调用：模型不断请求调用工具，导致死循环。

追问：

- 如何在 UI 上向用户展示“正在调用工具”的中间状态？

7. 你们项目中是如何处理 AI 生成内容的持久化和多端同步的？

难度：★★

考点：数据架构、同步机制

答案要点：

通常采用本地存储与云端同步相结合的方案，确保响应速度和数据一致性。

- 本地缓存：使用 IndexedDB 存储历史对话，实现秒开体验。
- 增量同步：仅同步新增的消息 chunk，减少网络开销。
- 冲突处理：基于时间戳或 Vector Clock 处理多端编辑冲突。
- 隐私保护：对敏感对话内容进行本地加密存储。

常见坑：

- 存储空间溢出（LocalStorage 仅 5MB）。
- 切换设备后，流式生成的消息显示不完整。

8. 谈谈 Web Worker 在 AI 前端应用中的实际应用场景。

难度：★★

考点：多线程、性能

答案要点：

Web Worker 主要用于处理计算密集型任务，防止主线程卡顿。

- 文本处理：大文档的切片（Chunking）、清洗和正则匹配。
- 本地模型运行：在 Worker 中运行 Transformers.js 等库进行 Embedding 或推理。
- 复杂渲染：对超长 Markdown 或代码高亮的预解析。
- 实时加密：对上传的私有文档进行端到端加密。

常见坑：

- Worker 与主线程频繁通信导致的序列化开销。
- 在 Worker 中无法直接操作 DOM 的限制。

三面（架构与综合能力）

1. 如果让你从零设计一个企业级 AI Copilot 的前端架构，你会如何考虑技术选型和模块划分？

难度：★★★

考点：系统架构、技术选型

答案要点：

架构设计应遵循高内聚低耦合原则，重点解决 AI 交互的特殊性。

- 核心分层：交互层（Chat UI）、协议层（SSE/WS/Fetch）、模型适配层（不同 LLM 供应商适配）、状态管理层（上下文维护）。
- 插件系统：设计一套标准的 Plugin SDK，允许业务方快速接入自定义工具（Tool Use）。
- 性能监控：集成 Token 消耗统计、首字延迟（TTFT）、渲染帧率监控。
- 安全策略：内置内容安全策略（CSP）、敏感词拦截、请求签名机制。

常见坑：

- 架构过于绑定某个特定的模型 API。
- 忽略了国际化（i18n）在 AI 生成内容中的复杂性。

追问：

- 如何在架构层面支持“多模态”输入（图片、语音、文件）？

2. AI 领域技术迭代极快，你如何保证前端团队在快速跟进新技术的同时，维持系统的稳定性？

难度：★★★

考点：技术管理、工程素养

答案要点：

通过抽象屏蔽差异，通过自动化测试保障底线。

- 接口抽象：建立统一的模型网关层，前端只对接标准协议，后端负责模型切换。
- 自动化评测：引入 E2E 测试模拟对话流，利用 LLM-as-a-Judge 对前端渲染结果进行回归测试。
- 灰度发布：针对模型升级或 Prompt 变更，采用 A/B Testing 观察用户反馈指标。
- 持续学习：建立团队内部的知识库，定期复盘 AI 落地中的坑点。

常见坑：

- 盲目追求新技术导致线上代码频繁重构，稳定性下降。

3. 在 AI 落地业务的过程中，你遇到过哪些关于“用户体验”的挑战？你是如何解决的？

难度：★★★

考点：产品思维、问题解决

答案要点：

AI 应用的挑战在于其“不确定性”，前端需要通过设计手段将其转化为确定性。

- 消除焦虑：针对模型响应慢的问题，设计多阶段 Loading（思考中 -> 检索中 -> 组织语言中）。

- 引导预期：通过 Prompt Suggestion（建议词）引导用户提问，减少无效交互。
- 容错设计：提供“重新生成”、“手动修改 Prompt”等功能，允许用户纠偏。
- 幻觉处理：在 UI 上显著标注 AI 生成内容的参考来源，降低幻觉带来的误导。

常见坑：

- 只有冷冰冰的加载动画，用户不知道 AI 到底在干什么。

4. 谈谈 Web AI 的未来趋势，例如 WebGPU 对前端 AI 应用会带来什么改变？

难度：★★★

考点：技术视野

答案要点：

WebGPU 将极大提升浏览器端的算力，使“端侧模型”成为可能。

- 算力飞跃：相比 WebGL，WebGPU 更接近底层硬件，能更高效地进行张量运算。
- 隐私与成本：模型在本地运行，无需上传数据到服务端，既保护隐私又节省服务器 GPU 成本。
- 离线能力：支持完全离线的 AI 交互，如本地文档摘要、图像处理。
- 混合架构：未来可能是“云端大模型 + 端侧小模型”的协同模式。

常见坑：

- 忽略了移动端设备对 WebGPU 支持程度的差异。

5. 如何评估一个 AI 前端项目的成功？你会关注哪些核心指标？

难度：★★

考点：数据驱动、业务价值

答案要点：

评价指标应结合技术性能与业务转化。

- 技术指标：TTFT（首字响应时间）、生成完成总耗时、流式中断率、渲染卡顿率。
- 业务指标：会话轮数（Engagement）、采纳率（如代码补全的 Accept Rate）、任务完成率。
- 成本指标：Token 消耗量、API 调用成功率。
- 用户反馈：点赞/点踩比、用户留存。

常见坑：

- 只关注技术指标，忽略了 AI 是否真的解决了用户问题。

6. 聊聊你在过去一年中遇到的最难的一个 AI 相关技术问题，你是如何排查并解决的？

难度：★★★

考点：综合能力、抗压能力

答案要点：

(此题为开放性问题，需结合实际经验)

- 问题描述：清晰定义问题的现象、环境和影响范围。
- 排查路径：使用了哪些工具（如 Chrome DevTools, Wireshark, 性能分析器）。
- 核心矛盾：发现问题的底层原因（如内存泄漏、协议不匹配、Prompt 歧义）。
- 解决方案：对比多个方案后的最终选择，以及上线后的效果。
- 经验沉淀：如何避免同类问题再次发生。

常见坑：

- 叙述过于琐碎，没有重点。
- 无法说明自己在解决问题中的核心贡献。